

uni-app 中 swiper（滑块视图容器）用法详解（适配 uni-app 2.3 版本）

uni-app 中 swiper（滑块视图容器）用法详解（适配 uni-app 2.3 版本）

`swiper` 是 uni-app 核心组件之一，用于实现轮播图、滑动切换页面/模块等效果，兼容小程序、App、H5 等多端。以下从**基础用法**、**核心属性**、**事件**、**常见场景**等维度全面讲解：

一、基础结构（必看）

`swiper` 是容器，内部必须嵌套 `swiper-item` 作为滑块子项（每个 `swiper-item` 对应一个滑动页），且一个 `swiper` 内只能有 `swiper-item` 直接子元素。

最简示例（轮播图/基础滑动）

Code block

```
1  <template>
2    <!-- swiper 容器 -->
3    <swiper class="swiper-container" indicator-dots autoplay interval="3000">
4      <!-- 滑块1 -->
5      <swiper-item>
6        <view class="swiper-item bg-red">滑块1</view>
7      </swiper-item>
8      <!-- 滑块2 -->
9      <swiper-item>
10       <view class="swiper-item bg-blue">滑块2</view>
11     </swiper-item>
12     <!-- 滑块3 -->
13     <swiper-item>
14       <view class="swiper-item bg-green">滑块3</view>
15     </swiper-item>
16   </swiper>
17 </template>
18
19 <style scoped>
20   .swiper-container {
21     width: 100%;
22     height: 200px; /* 必须设置高度，否则容器无显示 */
23   }
24   .swiper-item {
25     width: 100%;
```

```
26 height: 100%;
27 display: flex;
28 justify-content: center;
29 align-items: center;
30 color: #fff;
31 font-size: 20px;
32 }
33 .bg-red { background: #f00; }
34 .bg-blue { background: #00f; }
35 .bg-green { background: #0f0; }
36 </style>
```

二、核心属性（uni-app 2.3 版本重点）

`swiper` 的属性决定滑动行为、视觉样式，以下是高频常用属性，附关键说明：

属性名	类型	默认值	说明（2.3 版本适用）
<code>indicator-dots</code>	Boolean	false	是否显示面板指示小圆点
<code>indicator-color</code>	String	rgba(0,0,0,.3)	指示点未选中时的颜色
<code>indicator-active-color</code>	String	#000000	指示点选中时的颜色
<code>autoplay</code>	Boolean	false	是否自动切换
<code>interval</code>	Number	5000	自动切换时间间隔，需配合 <code>autoplay</code> 使用
<code>duration</code>	Number	500	滑动动画时长（ms）
<code>circular</code>	Boolean	false	是否采用衔接滑动播，最后一页滑到白）
<code>vertical</code>	Boolean	false	是否纵向滑动（默认横向）
<code>current</code>	Number	0	当前显示滑块的索引，支持 <code>v-model</code> 双向绑定，支持 <code>v-model</code> 简单模式
<code>previous-margin</code>	String	"0px"	前边距，可露出前一部分（如 "20px"）

			合 next-margin 式轮播)
next-margin	String	"0px"	后边距，可露出后一部分
display-multiple-items	Number	1	同时显示的滑块数持，需设置 width 用于多列滑动)
disable-touch	Boolean	false	是否禁止用户触摸留自动切换/代码控

属性示例（卡片式轮播）

Code block

```
1  <swiper
2    class="swiper-container"
3    indicator-dots
4    circular
5    previous-margin="20px"
6    next-margin="20px"
7  >
8    <swiper-item v-for="(item, index) in 5" :key="index">
9      <view class="card-item">卡片{{ index + 1 }}</view>
10   </swiper-item>
11 </swiper>
12
13 <style scoped>
14 .swiper-container {
15   height: 150px;
16   padding: 10px 0;
17 }
18 .card-item {
19   width: 100%;
20   height: 100%;
21   background: #409eff;
22   border-radius: 8px;
23   color: #fff;
24   display: flex;
25   justify-content: center;
26   align-items: center;
27 }
28 </style>
```

三、核心事件（交互控制）

通过事件监听滑动状态、切换行为，实现自定义逻辑：

事件名	说明	回调参数
change	当前滑块切换时触发（自动/手动切换均触发）	{ detail: { current: 索引 } }
touchstart	用户触摸滑块时触发	原生触摸事件对象
touchend	用户触摸结束时触发	原生触摸事件对象
transition	滑动动画过程中触发（2.3 版本支持）	{ detail: { dx, dy } } （滑动偏移）
animationfinish	滑动动画结束时触发（比 change 晚，仅动画完成后触发）	{ detail: { current: 索引 } }

事件示例（监听切换、手动控制）

Code block

```
1  <template>
2    <swiper
3      class="swiper-container"
4      :current="activeIndex"
5      @change="onSwiperChange"
6      @animationfinish="onAnimationFinish"
7    >
8      <swiper-item v-for="(item, index) in 3" :key="index">
9        <view class="swiper-item">滑块{{ index + 1 }}</view>
10     </swiper-item>
11   </swiper>
12   <!-- 手动切换按钮 -->
13   <view class="btn-group">
14     <button @click="switchTo(0)">切到第1页</button>
15     <button @click="switchTo(1)">切到第2页</button>
16     <button @click="switchTo(2)">切到第3页</button>
17   </view>
18 </template>
19
20 <script setup>
21 import { ref } from "vue";
22 // 当前激活的滑块索引
```

```
23  const activeIndex = ref(0);
24
25  // 滑块切换时触发
26  const onSwiperChange = (e) => {
27    console.log("当前滑块索引: ", e.detail.current);
28    activeIndex.value = e.detail.current;
29  };
30
31  // 滑动动画结束触发
32  const onAnimationFinish = (e) => {
33    console.log("滑动动画完成, 当前索引: ", e.detail.current);
34  };
35
36  // 手动切换滑块
37  const switchTo = (index) => {
38    activeIndex.value = index;
39  };
40  </script>
```

四、常见实战场景

场景1: 图片轮播（最常用）

结合 `image` 组件实现轮播图，适配多端：

Code block

```
1  <swiper
2    class="banner-swiper"
3    indicator-dots
4    autoplay
5    circular
6    interval="3000"
7  >
8    <swiper-item v-for="(img, index) in bannerList" :key="index">
9      <image
10        :src="img"
11        mode="widthFix"
12        class="banner-img"
13        @click="goToDetail(index)"
14      ></image>
15    </swiper-item>
16  </swiper>
17
18  <script setup>
19  const bannerList = ref([
20    "https://xxx.com/banner1.png",
```

```
21     "https://xxx.com/banner2.png",
22     "https://xxx.com/banner3.png"
23   ]);
24   // 点击轮播图跳转
25   const goToDetail = (index) => {
26     uni.navigateTo({ url: `/pages/detail/detail?id=${index + 1}` });
27   };
28 </script>
29
30 <style scoped>
31   .banner-swiper {
32     width: 100%;
33     height: 300rpx; /* 小程序/App 推荐用 rpx 适配 */
34   }
35   .banner-img {
36     width: 100%;
37     height: 100%;
38   }
39 </style>
```

场景2：纵向滑动（如垂直轮播公告）

Code block

```
1   <swiper
2     class="notice-swiper"
3     vertical
4     autoplay
5     interval="2000"
6     circular
7   >
8     <swiper-item v-for="(notice, index) in noticeList" :key="index">
9       <view class="notice-item">{{ notice }}</view>
10    </swiper-item>
11  </swiper>
12
13  <script setup>
14    const noticeList = ref([
15      "公告1: xxx活动上线",
16      "公告2: xxx功能优化",
17      "公告3: xxx维护通知"
18    ]);
19  </script>
20
21  <style scoped>
22    .notice-swiper {
```

```

23     width: 100%;
24     height: 80rpx;
25     background: #ffffbe8;
26 }
27 .notice-item {
28     width: 100%;
29     height: 100%;
30     line-height: 80rpx;
31     padding: 0 20rpx;
32     color: #ed6a0c;
33 }
34 </style>

```

场景3：多滑块同时显示（分页滑动）

通过 `display-multiple-items` 实现一页显示多个滑块：

Code block

```

1  <swiper
2    class="multi-swiper"
3    :display-multiple-items="2"
4    indicator-dots
5  >
6    <swiper-item v-for="(item, index) in 6" :key="index">
7      <view class="multi-item">商品{{ index + 1 }}</view>
8    </swiper-item>
9  </swiper>
10
11 <style scoped>
12 .multi-swiper {
13     width: 100%;
14     height: 200rpx;
15 }
16 .multi-item {
17     width: 100%;
18     height: 100%;
19     background: #f5f5f5;
20     margin: 0 10rpx;
21     display: flex;
22     justify-content: center;
23     align-items: center;
24 }
25 </style>

```

五、2.3 版本注意事项 & 避坑指南

1. **高度必设**: `swiper` 容器必须显式设置 `height` (百分比/固定值/rpx) , 否则会因高度为 0 无法显示;
2. **current 绑定**: 2.3 版本支持 `v-model:current` 或简写 `v-model` , 但部分端 (如微信小程序) 需用 `:current` + `@change` 手动同步;
3. **circular 兼容**: 开启 `circular` 循环轮播时, 若滑块数量 ≤ 1 , 会自动失效;
4. **性能优化**: 轮播图图片建议预加载, 或使用 `lazy-load` (`image` 组件属性), 避免卡顿;
5. **端差异**:
 - App/H5: 支持 `previous-margin/next-margin` 的百分比值 (如 "10%") ;
 - 小程序: 仅支持像素值 (如 "20px") , 需适配;
 - 纵向滑动在部分小程序端的 `indicator-dots` 样式需自定义调整。

六、扩展：自定义指示点/控制按钮

若默认指示点样式不满足需求, 可隐藏原生指示点, 自定义实现:

Code block

```
1  <view class="custom-swiper-wrap">
2    <!-- 隐藏原生指示点 -->
3    <swiper
4      class="custom-swiper"
5      :current="activeIdx"
6      @change="onChange"
7      circular
8    >
9      <swiper-item v-for="(item, index) in 3" :key="index">
10        <view class="swiper-item">滑块{{ index + 1 }}</view>
11      </swiper-item>
12    </swiper>
13    <!-- 自定义指示点 -->
14    <view class="custom-dots">
15      <view
16        class="dot"
17        :class="{ active: activeIdx === index }"
18        v-for="(item, index) in 3"
19        :key="index"
20        @click="activeIdx = index"
21      ></view>
22    </view>
23  </view>
24
25  <script setup>
26    import { ref } from "vue";
```



```
27  const activeIdx = ref(0);
28  const onChange = (e) => {
29    activeIdx.value = e.detail.current;
30  };
31  </script>
32
33  <style scoped>
34    .custom-swiper-wrap {
35      position: relative;
36    }
37    .custom-swiper {
38      width: 100%;
39      height: 200px;
40    }
41    .custom-dots {
42      position: absolute;
43      bottom: 10px;
44      left: 0;
45      right: 0;
46      display: flex;
47      justify-content: center;
48      gap: 8px;
49    }
50    .dot {
51      width: 8px;
52      height: 8px;
53      border-radius: 50%;
54      background: #ccc;
55    }
56    .dot.active {
57      background: #409eff;
58      width: 20px;
59      border-radius: 4px;
60    }
61  </style>
```

以上是 uni-app 2.3 版本 `swiper` 的核心用法，覆盖基础、进阶场景及避坑点，可直接复用至项目中。

(注：文档部分内容可能由 AI 生成)